

Nexus: A GPU Cluster Engine for Accelerating DNN-Based Video Analysis

Haichen Shen
Amazon Web Services

Lequn Chen
University of Washington

Yuchen Jin
University of Washington

Liangyu Zhao
University of Washington

Bingyu Kong
Shanghai Jiao Tong University

Matthai Philipose
Microsoft Research

Arvind Krishnamurthy
University of Washington

Ravi Sundaram
Northeastern University

Abstract

We address the problem of serving Deep Neural Networks (DNNs) efficiently from a cluster of GPUs. In order to realize the promise of very low-cost processing made by accelerators such as GPUs, it is essential to run them at sustained high utilization. Doing so requires cluster-scale resource management that performs detailed scheduling of GPUs, reasoning about groups of DNN invocations that need to be co-scheduled, and moving from the conventional whole-DNN execution model to executing fragments of DNNs. Nexus is a fully implemented system that includes these innovations. In large-scale case studies on 16 GPUs, when required to stay within latency constraints at least 99% of the time, Nexus can process requests at rates 1.8-12.7 $\times$ higher than state of the art systems can. A long-running multi-application deployment stays within 84% of optimal utilization and, on a 100-GPU cluster, violates latency SLOs on 0.27% of requests.

ACM Reference Format:

Haichen Shen, Lequn Chen, Yuchen Jin, Liangyu Zhao, Bingyu Kong, Matthai Philipose, Arvind Krishnamurthy, and Ravi Sundaram. 2019. Nexus: A GPU Cluster Engine for Accelerating DNN-Based Video Analysis. In *SOSP '19: Symposium on Operating Systems Principles, October 27–30, 2019, Huntsville, ON, Canada*. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3341301.3359658>

1 Introduction

Consider a cloud-scale video analysis service that allows thousands of tenants to analyze thousands of streams each concurrently. Increasingly, the core computations for this

workload are Deep Neural Networks (DNNs), which are networks of dense linear algebra computations. Specialized hardware accelerators for DNNs, in the form of Graphic Processing Units (GPUs, which this paper focuses on) and even more specialized Tensor Processing Units (TPUs) have emerged in the recent past. GPU accelerators process DNNs orders of magnitude faster and cheaper than CPUs in many cases. However, GPUs are expensive and very-high-capacity: modern devices *each* provide over 100 TFLOPS. Cost-savings from using them depends critically on operating them at sustained high utilization. A fundamental problem, therefore, is to distribute the large incoming workload onto a cluster of accelerators at high accelerator utilization and acceptable latency. We address this problem in this paper.

Conceptually, this problem can be thought of as sharding inputs via a distributed frontend onto DNNs on backend GPUs. Several interacting factors complicate this viewpoint. First, given the size of GPUs, it is often necessary to place different types of networks on the same GPU. It is then important to select and schedule them so as to maximize their combined throughput while satisfying latency bounds. Second, many applications consist of *groups* of DNNs that feed into each other. It is important to be able to specify these groups and to schedule the execution of the entire group on the cluster so as to maximize performance. Third, it is well known that dense linear algebra computations such as DNNs execute much more efficiently when their inputs are *batched* together. Batching complicates scheduling and routing because (a) it benefits from cross-tenant and cross-request coordination and (b) it forces the underlying bin-packing-based scheduling algorithms to incorporate batch size. Fourth, the increasingly common use of *transfer learning* in today's workloads has led to specialization of networks, where two tasks that formerly used identical networks now use networks that are only mostly identical. Since batching works only when multiple inputs are applied to the *same* model in conventional DNN execution systems, the benefits of batching are lost.

Nexus is a GPU cluster for DNN execution that addresses these problems to attain high execution throughput under

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SOSP '19, October 27–30, 2019, Huntsville, ON, Canada

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-6873-5/19/10...\$15.00

<https://doi.org/10.1145/3341301.3359658>

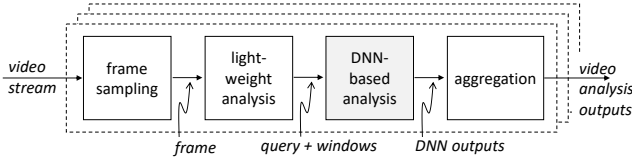


Figure 1: A typical vision processing pipeline. Nexus is designed to provide DNN-based analysis for tens of thousands of streams.

latency Service Level Objectives (SLOs). It uses three main techniques to do so. First, it relies on a novel *batching-aware scheduler* (Section 6.1) that performs bin packing when the balls being packed into bins have variable size, depending on the size of the batch they are in. This schedule specifies the GPUs needed, the distribution of DNNs across them, and the order of their execution so as to maximize execution throughput while staying within latency bounds. Second, it allows groups of related DNN invocations to be written as *queries* and provides automated query optimization to assign optimal batch sizes to the components of the query so as to maximize overall execution throughput of the query while staying within its latency bounds (Section 6.2). Finally, Nexus breaks from orthodoxy and allows batching of *parts of networks* with different batch sizes. This enables the batched execution of specialized networks (Section 6.3).

Nexus is completely implemented as a containerized system deployable on a commercial cloud and comprises of roughly 10k lines of C++. We have deployed Nexus on a 100-GPU cluster. On focused 16-GPU experiments compared with existing DNN serving systems (Tensorflow Serving [25] and Clipper [6]), we measure the maximal request rate processed by these systems on fixed applications such that at least 99% of requests are handled within latency SLOs. By this metric, Nexus is able to handle 1.8-4.4 $\times$ more requests on a traffic monitoring application, and 9.4-12.7 $\times$ on a game-stream analysis case study. On a much larger experiment on an 100-GPU cluster, 7 applications and 12 different models, Nexus achieves a maximal request rate of over 98.7% while maintaining similar high throughputs.

2 Background

A vision-based application aggregates visual information from one or more video streams using custom “business” logic. Each stream is processed using a pipeline similar to that in Figure 1. CPU-based code, either on the edge or in the cloud, selects *frames* from the stream for processing, applies business logic to identify what parts (or *windows*) of the image need deeper analysis, applies a DNN *query* to these windows, and aggregates the results in an application-specific way, often writing to a database. A query may represent a single DNN applied to the window, but often it may represent a sequence of dependent DNN applications, e.g., running an object detector on the window and running a car make/model detector on all sub-windows containing cars.

Model	CPU lat. (ms)	GPU lat. (ms)	CPU cost (\$) (0.1TF peak)	TPU cost (\$) (180TF peak)	GPU cost (\$) (125TF peak)
Lenet5	6	<0.1	\$0.01	\$0.00	\$0.00
VGG7	44	<1	0.13	0.01	0.01
Resnet50	1130	6.2	4.22	0.48	0.12
Inception4	2110	7.0	8.09	0.93	0.23
Darknet53	7210	26.3	24.74	2.85	0.70

Table 1: DNN execution latencies and estimated costs per 1000 invocations.¹Acceleration may be necessary to meet latency deadlines, but can also be cheaper, given low cost/TFLOPS.

Typically, a stream is sampled a few times a second or minute, and the DNN query should complete execution in tens to hundreds of milliseconds (for “live” applications) or within several hours for (“batch” applications). The execution of DNNs dominates the computation pipeline, and the cost of executing them dominates the cost of the vision service. Nexus provides a standalone service that implements the DNN-based analysis stage for vision pipelines.

2.1 Accelerators and the challenge of utilizing them

As Table 1 shows, a key to minimizing the cost of executing DNNs is the use of specialized accelerators such as GPUs and TPUs, which are highly optimized to execute the dense linear algebra computations that comprise DNN models. The table shows the execution latency and the dollar cost of 1000 invocations for a few common models on CPUs and GPUs. Execution times on CPUs can be orders of magnitude slower than that on GPUs. For many applications, therefore, latency constraints alone may dictate GPU-accelerated execution.

Perhaps more fundamentally, GPUs and TPUs promise much lower cost per operation than even highly accelerated CPUs: Table 1 lower-bounds the cost of executing a model by assuming that models can be executed at peak speed on each platform. Even compared to state of the art CPUs, accelerators can yield a cost advantage of up to 9 $\times$ (for TPUs) and 34 $\times$ (for GPUs). On the other hand, accelerators have extremely high computational capacities (e.g., 125 TFLOPS for the NVIDIA V100). *To realize their cost savings, it is critical to sustain high utilization of this capacity.* Sustaining high utilization is hard, however. For instance, the LeNet model of Table 1 consumes 20 MOPs to run, implying that *a single V100 would require 125 TFLOPS $\div$ 20 MOPs = 6.25M inputs/second to run at full utilization!*

No single stream, or even most applications, can yield such rates. By aggregating inputs across streams and applications, Nexus is designed to funnel adequate work to each accelerator. However, as we discuss next, having “enough” work is

¹Per-device prices for 1000 invocations assuming peak execution rates on on-demand instances of AWS c5.large (Intel AVX 512), p2.xlarge (NVIDIA K80), p3.2xlarge (NVIDIA V100) and GCP Cloud TPU.

not sufficient to achieve high utilization: it is important to group the right type of work in the right place.

2.2 Placing, packing and batching DNNs

DNNs are networks of dense linear algebra operations (e.g., matrix multiplication and convolution), called *layers* or *kernels*. Networks are also called *models*. By default, the GPU simply executes the kernels presented to it in the order received. The kernels themselves are often computationally intensive, requiring MFLOPs to GFLOPs to execute, and range in size from one MB to hundreds of MBs. These facts have important implications for GPU utilization.

First, loading models into memory can cost hundreds of milliseconds to seconds. When serving DNNs at high volume, therefore, it is usually essential to *place* the DNN on a particular GPU by pre-loading it on to GPU memory and then re-using it across many subsequent invocations. Placement brings with it the traditional problems of efficient *packing*. Which models should be co-located on each GPU, and how should they be scheduled to minimize mutual interference?

Second, it is well known that the processor utilization achieved by kernels depends critically upon *batching*, i.e., grouping input matrices into higher-dimensional ones before applying custom “batched” implementations of the kernels. Intuitively, batching allows kernels to avoid stalling on memory accesses by operating on each loaded input many more times than without batching. On an NVIDIA GTX1080, batching improves the throughput of model execution by 4.7-13.3× for batch sizes of 32 for VGG, ResNet, and Inception models relative to executing them individually. Further, our empirical measurements indicate that we can often use a linear model to fit the batched execution latency as follows:

$$\text{batch_lat}(b) = \alpha b + \beta, \quad (1)$$

where β is the fixed cost to invoke a model and α is the cost of each additional task in the batch. Large batches amortize the fixed cost β and help achieve higher throughputs.

Although batching is critical for utilization, it complicates the resource allocation and scheduling decisions made inside of a cluster. We elaborate on these issues in Section 4. Further, batching is conventionally only feasible when the same model is invoked with different inputs. For instance, we expect many applications to use the same well-known, generally applicable, models (e.g., Resnet50 for object recognition). However, the generality of these models comes at the price of higher resource use. It has become common practice [12, 24] to use smaller models *specialized* (using “transfer learning”) to the few objects, faces, etc. relevant to an application by altering (“re-training”) just the output layers of the models. Since such customization destroys the uniformity required by conventional batching, making specialized models play well with batching is often critical to efficiency.

3 Related work

The projects most closely related to Nexus are Clipper [6] and Tensorflow Serving [25]. Clipper is a “prediction serving system” that serves a variety of machine learning models including DNNs, on CPUs and GPUs. Given a request to serve a machine learning task, Clipper selects the type of model to serve it, batches requests, and forwards the batched requests to a backend container. By batching requests, and adapting batch sizes online under a latency SLO, Clipper takes a significant step toward Nexus’s goal of maximizing serving throughput under latency constraints. Clipper also provides approximation and caching services, complementary to Nexus’s focus on executing all requests exactly but efficiently. Tensorflow Serving can be viewed as a variant of Clipper that does not provide approximation and caching, but also has additional machinery for versioning models.

To the basic batched-execution architecture of Clipper, Nexus builds along the dimensions of *scale*, *expressivity* and *granularity*. These techniques address the challenges brought up earlier in this section and thus reflect Nexus’s focus on executing DNNs on GPUs at high efficiency and scale.

Scale: Nexus provides the machinery to scale serving to large, changing workloads. In particular, it automates the allocation of GPU resources and the placement and scheduling of models across allocated resources. It provides a distributed frontend that scales with requests. These functions are performed on a continuing basis to adapt to workloads.

Expressivity: Nexus provides a query mechanism that (a) allows related DNN execution tasks to be specified jointly, and (b) allows the user to specify the latency SLO just at the whole-query level. Nexus then analyzes the query and allocates latency bounds and batch sizes to constituent DNN tasks so as to maximize the throughput of the whole query.

Granularity: Where Clipper limits the granularity of batched execution to whole models, Nexus automatically identifies common *subgraphs* of models and executes them in a batch. This is critical for batching on specialized models, which often share all but the output layer, as described previously.

Other work has explored system-level optimization for DNN inference. They address goals other than Nexus’s focus on improving accelerator utilization via aggressive batching and are broadly complementary. MCDNN [14] is an early example of a system that exploited shared model prefixes resulting from specialization on mobile CPUs. However, like Mainstream [19] after it, which provided a more sophisticated infrastructure for managing specialization on server CPUs, it focused on shared prefixes that applied to a *common* input, with the goal of avoiding redundant computation across prefixes. Nexus, on the other hand, focuses on shared prefixes that operate on *different* inputs with the goal of utilizing the underlying GPU hardware more efficiently via batching.

A common theme in these and other systems is to automatically select *approximately equivalent*, but more performant, models for a task. MCDNN [14] introduced the notion of selecting from a *catalog* of variants of models that trade off accuracy for performance. VideoStorm [42] generalized the notion of exploiting such tradeoffs to all parameters of the computer vision system, not just those related to DNNs. NoScope [20] proposed specializing models to queries to speed up query execution. More recent work [35] proposes cascading small dynamically specialized models that handle the common case with larger backup models. Focus [17] applies cascading to the video indexing scenario, backing up a small, frequent ingest-time indexing DNN with a large but infrequent query-time DNN. Nexus focuses on providing an engine for *non-approximating* execution of the incoming models, a facility that can potentially be used as a backend for these approximation-based systems.

Serving DNNs at scale is similar to other large-scale short-task serving problems. These systems have distributed front ends that dispatch low-latency tasks to queues on the backend servers. Sparrow [27] focuses on dispatch strategies to reduce the delays associated with queuing in such systems. Slicer [3] provides a fast, fault-tolerant service for dividing the back end into shards and load balancing across them. Both systems assume that the backend server allocation and task placement is performed at a higher (application) level, using cluster resource managers such as Mesos [16] or Omega [33]. Nexus shares the philosophy of these systems of having a fast data plane that dispatches incoming messages from the frontend to backend GPUs and a slower control plane that performs more heavyweight scheduling tasks, such as resource allocation, packing and load balancing. On the other hand, compared to these generic systems, Nexus provides query processing, task allocation, and sub-task scheduling functionality that is targeted to better batching over DNN-based workloads.

Much work has focused on producing faster models often at small losses in accuracy [2, 31, 41]. Further, models of varying accuracy can be combined to maintain high accuracy and performance [6, 14, 17, 20, 35]. Nexus views the optimization, selection, and combination of models as best done by the application, and provides no special support for these functions. Our mechanisms are also orthogonal to the scheduling, placement, and time-sharing mechanisms in training systems [13, 30, 38] since DNN serving has to be performed within tight latency SLOs while maintaining high utilization.

4 Scheduling problems in batched execution

Batched execution of models improve GPU utilization but also raises many challenges in determining how cluster resources are allocated to different applications and how to batch model invocations without violating latency constraints.

Model A			Model B			Model C		
Batch	Lat	Req/s	Batch	Lat	Req/s	Batch	Lat	Req/s
4	50	80	4	50	80	4	60	66.7
8	75	107	8	90	89	8	95	84
16	100	160	16	125	128	16	125	128

Table 2: Batching profiles for models used in the example. Lat is the latency (ms) for processing a batch, and Req/s is the throughput achieved.

Fundamentally, the algorithm for packing models on GPUs needs to take into account the fact that the processing cost of an input is “squishy”, i.e., it varies with the size of the batch within which that input is processed. Further, the latency of execution also depends on the batch size. This new version of bin packed scheduling, which we dub *squishy bin packing*, needs to reason explicitly about batching. Second, batching complicates query processing. If a certain latency SLO (Service Level Objective) is allocated to the query as a whole, the system needs to partition the latency across the DNN invocations that comprise the query so that each latency split allows efficient batched execution of the related DNN invocation. We call this *complex query scheduling*. Third, in addition to batching-aware resource allocation, the runtime dispatch engine also has to determine what requests are batched and what requests are dropped during periods of bursty arrival. We now use examples and measurements to elaborate on these underlying scheduling and resource allocation challenges.

4.1 Squishy bin packing

Consider a workload that consists of three different types of tasks that invoke different DNN models. Let the desired latency SLOs for tasks invoking models A, B, and C be 200ms, 250ms, and 250ms, respectively. Table 2 provides the batch execution latency and throughput at different batch sizes (i.e., the “batching profile”) for each model.

We first explore the basic scenario where all three types of tasks are associated with high request rates so that multiple GPUs are required to handle each task type. To maximize GPU efficiency, we need to choose the largest possible batch size while still meeting the latency SLO. Note that the batch execution cost for a given task type cannot exceed half of the task’s latency SLO; a task that missed being scheduled with a batch would be executed as part of the next batch, and thus its latency would be twice the batch execution cost. For example, the latency SLO for Model A tasks is 200 ms, so the maximum batch size we can use is 16. Therefore, the maximum throughput that Model A can achieve on a single GPU is 160 reqs/sec, and the number of GPUs to be allocated for Model A should be $r_A/160$, where r_A is the observed request rate. Similarly, the number of GPUs for models B and C should be $r_B/128$ and $r_C/128$, where r_B and r_C are the request rates for models B and C respectively. Figure 2(a) depicts the desired schedules for the different models.

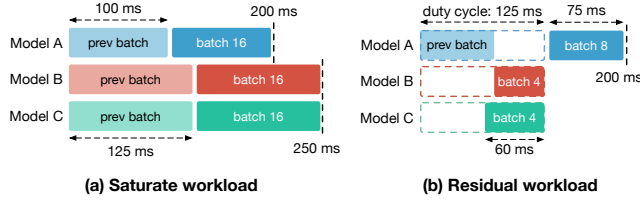


Figure 2: Resource allocation example.

We next consider a situation where the request rates for the models are lower, with each one requiring less than a GPU. In this case, the scheduler needs to consolidate multiple types of DNN tasks onto the same GPU to optimize resource utilization. Consider a workload where Model A receives 64 reqs/sec, and Model B and Model C receive 32 reqs/sec each. We consider schedules where multiple models are assigned to a GPU. The GPU then executes batches of different types of models in a round-robin manner, and it cycles through them over a time period that we refer to as the *duty cycle*. The worst-case latency for a task is no longer twice the batch execution cost but rather the sum of the *duty cycle* and the batch execution cost for that task type.

Given this setup, Model A tasks can be scheduled in batches of 8 as part of a duty cycle of 125ms; note that the resulting throughput is the desired rate of 64 reqs/sec, the batch execution cost for 8 tasks is 75ms, and the worst-case execution delay of 200ms matches the latency SLO (see Figure 2(b)). We then check whether the GPU has sufficient slack to accommodate tasks associated with models B or C. Within a duty cycle of 125ms, we would need to execute 4 tasks of either B or C to meet the desired rate of 32 reqs/sec. The batch execution cost of 4 model B tasks is 50ms, which can fit into the residual slack in the duty cycle. On the other hand, a batch of 4 model C tasks would incur 60ms and cannot be scheduled inside the duty cycle. Further, the worst-case latency for model B is the sum of the duty cycle and its own batch execution cost, 175ms(= 125 + 50), which is lower than its latency SLO 250ms. Thus, it is possible to co-locate Model B, but not Model C, on the same GPU as Model A.

We now make a few observations regarding the scenario discussed above and why the associated optimization problem cannot be addressed directly by known scheduling algorithms. First, unlike vanilla bin-packing that would pack fixed-size balls into bins, the tasks here incur lower costs when multiple tasks of the same type are squished together into a GPU. Second, in addition to the capacity constraints associated with the GPU's compute and/or memory capabilities, there are also latency constraints in generating a valid schedule. Third, there are many degrees of freedom in generating a valid schedule. The batch size associated with a model execution is not only a function of the request rate but also of the duty cycle in which the batch is embedded. In Section 6.1, we describe how to extend traditional algorithms to handle this setting.

4.2 Complex query scheduling

Applications often comprise of dependent computations of multiple DNN models. For example, a common pattern is a detection and recognition pipeline that first detects certain objects from the image and then recognizes each object. The developer will specify a latency SLO for the entire query, but since the system would host and execute the constituent models on different nodes, it would have to derive latency SLOs for the invoked models and the schedules that meet these latency SLOs. We discussed the latter issue in the previous example, and we now focus on the former issue.

Consider a query that executes Model X and feeds its output to Model Y. Suppose we have a 100ms latency budget for processing this query, and suppose that every invocation of X yields γ outputs (on average). When $\gamma < 1$, X operates as a filter; when $\gamma = 1$, X maps an input to an output; when $\gamma > 1$, X yields multiple outputs from an input (e.g., detection of objects within a frame).

Assume that Figure 3 depicts the batch execution latency and throughput of models X and Y. The system has to decide what latency SLOs it has to enforce on each model such that the overall latency is within 100ms and the GPU utilization of the query as a whole is maximized. For this example, we consider a limited set of latency split plans for models X and Y: (a) 40ms and 60ms, (b) 50ms and 50ms, (c) 60ms and 40ms. It would appear that plan (a) should work best since the sum of the throughputs is largest among the three plans, but a closer examination reveals some interesting details.

For workloads involving a large number of requests, let us assume that p and q GPUs execute X and Y, respectively. We then have $\gamma \cdot p \cdot T_X = q \cdot T_Y$, where T_X and T_Y are per-GPU throughputs of X and Y, such that the pipeline won't be bottlenecked by any model. We define the average throughput as the pipeline throughput divided by the total number of GPUs, which is $p \cdot T_X / (p + q)$. We evaluate the average throughputs for the three latency split plans with $\gamma = 0.1, 1, 10$. Figure 4 shows that each of the plans achieves the best performance for different γ values. In fact, there is no universal best split: it depends on γ , which can vary over time.

We note two observations from this example. First latency split for complex query impacts overall efficiency, and it is necessary to account both batch performance and workload statistics to make the best decision. Second, latency split should not be static but rather adapted over time in accordance with the current workload. Section 6.2 describes how Nexus automatically and continually derives latency splits.

4.3 Rate control and adaptive batching

Model serving systems need to perform adaptive batching based on the number of requests received. When there is a burst of requests, the system needs to drop certain requests in order to serve the remaining requests within the latency

Model X		Model Y	
Lat	Reqs/s	Lat	Reqs/s
40	200	40	300
50	250	50	400
60	300	60	500

Figure 3: Batch execution latency (ms) and throughput of models.

Latency budget		Avg Throughput (reqs/s)		
X	Y	$\gamma = 0.1$	$\gamma = 1$	$\gamma = 10$
40	60	192.3	142.9	40.0
50	50	235.3	153.8	34.5
60	40	272.7	150.0	27.3

Figure 4: The average throughput with three latency split plans for varying γ .

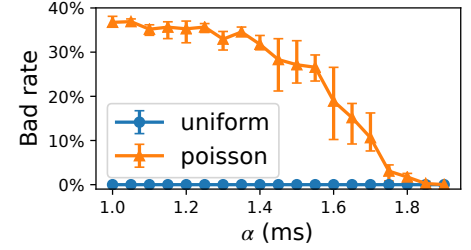


Figure 5: Performance of lazy dropping.

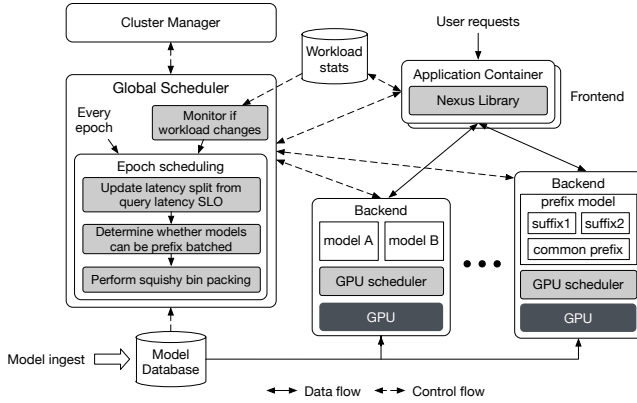


Figure 6: Nexus runtime system overview.

SLO. One approach is to perform lazy dropping, i.e., drop a request only when it has already missed its deadline, and determine the batch size based on the time budget remaining for the earliest request in the queue (as in Clipper [6]). We use simulation to evaluate this approach for different batching profiles (as modeled by Equation 1). We fix latency SLO to 100ms and optimal model throughput on a single GPU to 500 reqs/s, and vary α . Given the fixed throughput, the fixed cost of β reduces as we increase α . The workload is generated using uniform and Poisson arrivals with the mean request rate set to 90% of the optimal throughput. We define the *bad rate* to be the percentage of requests that exceed the deadline or get dropped. Figure 5 shows that the lazy dropping strategy performs poorly for Poisson distributions when α is small and β is correspondingly high. Since the system always attempts to execute the oldest received request, it often has to resort to a small batch size in order to meet the deadline, but this causes the dispatcher to fall behind further given the high fixed cost is not amortized over sufficient requests. This experiment indicates that even the runtime needs to consider batch efficiency in determining what tasks to dispatch.

5 Nexus architecture

The primary goal of Nexus is to attain high execution efficiency on GPU clusters while serving video analysis requests within a specified latency SLO. Our service model assumes that the system can drop requests if it cannot execute them within their deadlines (following prior work [5]). Note that

this is appropriate for video stream analysis, as the next sampled frame would typically be processed even if the earlier one is dropped. We also note that we could configure our system to simply delay the execution of requests that miss their deadlines to a later time and at a lower priority; many of our techniques would still be effective in improving efficiency in such a setting.

Nexus works on three planes (as depicted by Figure 6). The *management* plane allows developers to ingest and deploy applications and models, at a timescale of hours to weeks. The *control* plane, via the *global scheduler*, is responsible for resource allocation and scheduling at a typical timescale of seconds to minutes. The *data* plane, comprised of in-application *Nexus library* instances and backend modules (together, the *Nexus runtime*), dispatches and executes user requests at the timescale of milliseconds to seconds. The global scheduler interacts with the underlying cluster resource manager (e.g., Mesos [16], Azure Scale Sets [23]) to acquire CPUs/GPUs for the frontend/backend. A load balancer (not shown) from the underlying cluster spreads user requests across Nexus's distributed frontend. We sketch the three planes.

Management plane: Models are stored in a *model database* and may be accompanied by either a sample data set or a batching profile. Nexus uses the sample dataset, if available, to derive a batching profile. A profiler measures the execution latency and memory use for different batch sizes when the models are uploaded to Nexus. Applications are containers that provide the Nexus library to client programs. Developers store these *application containers* in cluster-provided container repositories and may instruct Nexus to ingest a container, at which point it is loaded from the repository onto a frontend CPU.

Control plane: The global scheduler is a cluster-wide resource manager that uses load statistics from the runtime. It uses this profile to add or remove frontend and backend nodes from the cluster, invokes the *epoch scheduler* to decide which models to execute and at what batch size, and which backend to place the models on so as to balance the load and maximize utilization. Multiple models may be mapped onto a single backend, albeit with an execution schedule that ensures they do not interfere as in Section 4.1. The mapping from models to backends is captured in a *routing table* that is

sent to frontends. The matching execution schedule for each backend is captured in a *schedule* that is sent to backends. On receiving a routing table, frontends update their current routing table. On receiving a schedule, backends load appropriate models into GPU memory and set their execution schedule.

Allocation, scheduling, and routing updates happen at the granularity of an *epoch*, typically 30-60s, although a new epoch can also be triggered by large changes in workload. To prevent oscillation from frequent reconfiguration, we limit the minimum period between two epochs to 10 seconds. Epoch scheduling involves the following:

- Produce an updated split of the latency SLO for the individual models inside a query (see Section 6.2).
- Combine two or more models that share a prefix and latency SLO into a new *prefix-batched* model (see Section 6.3).
- Perform profile-guided squishy bin packing to allocate the GPU resources for each model. (see Section 6.1).

Because epoch scheduling reacts to workload change at the granularity of an epoch at best, Nexus relies on admission control that drops excessive requests to make sure that most (targeted for 99% in the evaluation) requests can be served within their latency constraints within an epoch. Requests are dropped using an early-drop policy (see Adaptive Batching in Section 6.3).

Data plane: When a user request comes into (a replica of) an application container, the application invokes DNNs via the Nexus library API. The library consults the local routing table to find a suitable backend for that model, dispatches the request to the backend, and delivers responses back to the application. The application is responsible for packaging and delivering the end-result of the query to the user. A backend module uses multiple threads to queue requests from various frontends, selects and executes models on these inputs in a batched mode according to the current schedule, and sends back the results to frontends. It can utilize one or more GPUs on a given node, with each GPU managed by a *GPU scheduler* that schedules tasks on it.

6 Batch-aware scheduling and dispatch

We now describe the algorithms used by the global scheduler and the node dispatcher. First, we consider the case of scheduling streams of individual DNN task requests, given their expected arrival rates and latency SLOs. We next consider how to schedule streams of more complex queries/jobs that invoke multiple DNN tasks. We then describe how the node runtime cycles through DNNs and performs batching.

6.1 Scheduling streams of individual DNN tasks

We build upon the discussion presented in Section 4.1. The scheduler identifies for each cluster node the models hosted by it. As discussed earlier, the scheduling problem has the

Notation	Description
S_i	Session i
M_{k_i}	DNN model k_i for session S_i
L_i	Latency constraint for session S_i
R_i	Request rate for session S_i
$\ell_{k_i}(b)$	Execution cost for M_{k_i} and batch size b

Table 3: Notation

structure of bin-packing [21], but we need to address the "squishiness" of tasks and the need to meet latency SLOs.

Inputs: The scheduler is provided the request rate for a model at a given latency SLO. We refer to the requests for a given model and latency SLO as a *session*. Note that a *session* would correspond to classification requests from different users and possibly different applications that invoke the model with a given latency constraint. Table 3 describes the notation used below. Formally, a session S_i specifies a model M_{k_i} and a latency constraint L_i , and there is a request rate of R_i associated with it. The scheduler is also provided with the batching profiles of different models. The latency of executing b invocations of M_{k_i} is $\ell_{k_i}(b)$. We assume that throughput is non-decreasing with batch size b .

Scheduling overview: The scheduler allocates one or more sessions to each GPU and specifies their target batch sizes. Each GPU node n is then expected to cycle through the sessions allocated to it, execute invocations of each model in batched mode, and complete one entire cycle of batched executions within a *duty cycle* of d_n . For sessions that have a sufficient number of user requests, one or more GPU nodes are allocated to a single session. The integer programming formulation and a greedy approximation algorithm described below computes the residual workload for such sessions (after allocating an integral number of GPUs) and then attempts to perform bin packing with the remaining smaller sessions.

Scheduling Large Sessions: For session S_i , we first compute the peak throughput of M_{k_i} when executed in isolation on a GPU. With a batch size b , the worst case latency for any given request is $2\ell_{k_i}(b)$, as we explained in Section 4.1. Denote batch size B_i as the maximum value for b that meets the constraint $2\ell_{k_i}(b) \leq L_i$. Therefore, the maximal throughput, denoted by T_i , for session S_i on a single GPU is $B_i/\ell_{k_i}(B_i)$. The number of GPU nodes we allocate to execute just S_i requests is $n = \lfloor R_i/T_i \rfloor$. There will be a residual unallocated load for session S_i after taking into account this allocated load. Note that $n = 0$ for sessions that don't have sufficient requests to utilize an entire GPU. (Function SCHEDULESATURATE in Algorithm 1 provides the pseudocode.)

Optimization problem for scheduling residual workload: We next consider the problem of scheduling the residual loads, i.e., a workload where none of the models have sufficient load to need an entire GPU. The optimization problem can be expressed as an integer programming problem.

Decision Variables	Definition
$g_j \in \{0, 1\}$	Whether GPU j is in use
$s_{ij} \in \{0, 1\}$	Whether session i is assigned to GPU j
$b_{ij} \in \mathbb{R}_{\geq 0}$	Batch size of session i on GPU j

We need to minimize: $\sum_j g_j$, while subject to:

$$s_{ij} = 1 \rightarrow g_j = 1 \quad \forall j \quad (a)$$

$$\sum_j s_{ij} = 1 \quad \forall i \quad (b)$$

$$s_{ij} = 0 \rightarrow b_{ij} = 0 \quad \forall i, j \quad (c)$$

$$s_{ij} = 1 \rightarrow b_{ij} \geq 1 \quad \forall i, j \quad (d)$$

$$d_j = \sum_{i: s_{ij}=1} \ell_{k_i}(b_{ij}) \quad \forall j \quad (e)$$

$$d_j + \ell_{k_i}(b_{ij}) \leq L_i \quad \forall i, j (s_{ij} = 1) \quad (f)$$

$$b_{ij} \geq r_i d_j \quad \forall i, j (s_{ij} = 1) \quad (g)$$

The constraints correspond to the following requirements.

- (a) Sessions can only be assigned to GPUs that are in use.
- (b) Each session can only be assigned to one GPU.
- (c) b_{ij} is 0 if i is not assigned to GPU j .
- (d) b_{ij} is at least 1 if i is assigned to GPU j .
- (e) Length of a duty cycle as a function of assigned sessions.
- (f) Latency SLO constraint.
- (g) Scheduled rate meets the request rate requirement.

Note that some of the constraints are not linear, and we omit details on how to express them in a strictly linear way. We used the CPLEX package to solve this formulation on benchmark workloads. Even after incorporating optimizations, such as using a greedy algorithm to provide both an initial feasible solution and an upper bound for the number of GPUs needed, solving the integer program is expensive. For example, computing the minimum number of GPUs for 25 sessions takes several hours, even though the upper bound, determined via a greedy algorithm, is 8 GPUs. Further this optimization problem is proven to be strongly NP-hard (see Appendix A). We, therefore, resort to the following greedy scheduling algorithm.

Greedy scheduling algorithm for residual loads: For the bin packing process, the scheduler inspects each residual session in isolation and computes the largest batch size and the corresponding duty cycle in order to meet the throughput and SLO needs. The intuition behind choosing the largest batch size is to have an initial schedule wherein the GPU operates at the highest efficiency. This initial schedule, however, is not cost-effective as it assumes that each GPU is running just one session within its duty cycle, so the algorithm then attempts to merge multiple sessions within a GPU's duty cycle. In doing so, it should not violate the latency SLOs, so we require that the merging process only reduces the duty cycle of the combined allocation. The algorithm considers sessions in decreasing order of associated work and merges

Algorithm 1 Squishy Bin Packing Algorithm

```

SQUISHYBINPACKING(Sessions)
1: nodes, residue_loads  $\leftarrow$  SCHEDULESATURATE(Sessions)
2: nodes  $\leftarrow$  nodes  $\oplus$  SCHEDULERESIDUE(residue_loads)
3: return nodes

SCHEDULESATURATE(Sessions)
4: nodes, residue_loads  $\leftarrow$  [], []
5: for  $S_i = \langle M_{k_i}, L_i, R_i \rangle$  in Sessions do
6:    $B_i \leftarrow \operatorname{argmax}_b (2\ell_{k_i}(b) \leq L_i)$ 
7:    $T_i \leftarrow B_i / \ell_{k_i}(B_i)$ 
8:   let  $R_i = n \cdot T_i + r_i$ 
9:   nodes  $\leftarrow$  nodes  $\oplus$   $n$  GPU nodes for  $M_{k_i}$  with batch  $B_i$ 
10:  residue_loads  $\leftarrow$  residue_loads  $\oplus \langle M_{k_i}, L_i, r_i \rangle$ 
11: return nodes, residue_loads

SCHEDULERESIDUE(residue_loads)
12: for  $\langle M_{k_i}, L_i, r_i \rangle$  in residue_loads do
13:    $b_i \leftarrow \operatorname{argmax}_b (\ell_{k_i}(b) + b/r_i \leq L_i)$ 
14:    $d_i \leftarrow b_i / r_i$ 
15:    $occ_i \leftarrow \ell_{k_i}(b_i) / d_i$ 
16: sort residue_loads by  $occ_i$  in descending order
17: nodes  $\leftarrow$  []
18: for  $\langle M_{k_i}, L_i, r_i, b_i, d_i, occ_i \rangle$  in residue_loads do
19:   max_occ  $\leftarrow$  0
20:   max_node  $\leftarrow$  NULL
21:   for  $n = \langle b, d, occ \rangle$  in nodes do
22:      $n' \leftarrow \operatorname{MERGENODES}(n, \langle b_i, d_i, occ_i \rangle)$ 
23:     if  $n' \neq \text{NULL}$  and  $n'.occ > \text{max\_occ}$  then
24:       max_occ  $\leftarrow$   $n'.occ$ 
25:       max_node  $\leftarrow$   $n'$ 
26:   if max_node  $\neq$  NULL then
27:     replace max_node for its original node in nodes
28:   else
29:     nodes  $\leftarrow$  nodes  $\oplus \langle b_i, d_i, occ_i \rangle$ 
30: return nodes

```

them into existing duty cycles that have the highest allocations, thus following the design principle behind the best-fit decreasing technique for traditional bin packing.

We now elaborate on this greedy scheduling algorithm (which is also depicted in function SCHEDULERESIDUE of Algorithm 1). Denote r_i to be the request rate of a residual load. Suppose we execute the residual load with batch size b , the duty cycle d for gathering b inputs is b/r_i . Then, the worst case latency is $d + \ell_{k_i}(b)$. Therefore, we have the constraint:

$$d + \ell_{k_i}(b) = b/r_i + \ell_{k_i}(b) \leq L_i \quad (2)$$

We begin residual load scheduling (Line 12-15) by choosing for session S_i the maximum batch size b_i that satisfies the above constraint. The corresponding duty cycle d_i is also at its maximal value. Denote *occupancy* (occ) as the fraction of the duty cycle d_i occupied by S_i 's residual load invocations: $occ_i(b_i) = \ell_{k_i}(b_i) / d_i$.

Next, we start to merge these fractional GPU nodes into fewer nodes (Line 16-30 in Algorithm 1). This part resembles

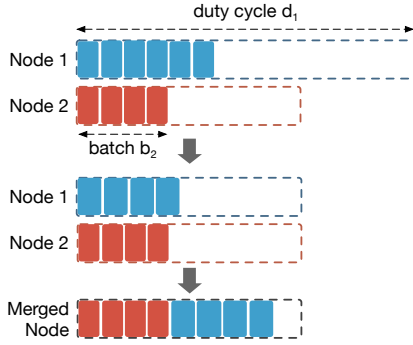


Figure 7: Merge two nodes into one. Use the smaller duty cycle as new duty cycle for both nodes. Update the batch size accordingly and re-estimate the batch latency. If sum of latencies doesn't exceed new duty cycle, the two nodes can be merged.

the classic bin packing algorithm that first sorts sessions by decreasing occupancy and merges two nodes into a single node by best fit. The primary difference is how to determine whether two nodes can be merged such that their sessions won't violate the latency SLOs. Figure 7 depicts the process of merging two nodes. Suppose we have two sessions S_1 and S_2 on separate nodes, with request rates r_1 and r_2 , assigned batch sizes b_1 and b_2 , and duty cycles d_1 and d_2 . We use $d = \min(d_1, d_2)$ as the new duty cycle of a combined node. Without loss of generality, we assume $d = d_2$. We then use $b'_1 = d \cdot r_1 \leq b_1$ as the new batch size for S_1 . Note that the worst case latency of requests in S_1 now becomes $d + \ell_{k_1}(b'_1) \leq d_1 + \ell_{k_1}(b_1) \leq L_i$, and we won't violate the latency constraint for S_1 by this adjustment. If $\ell_{k_1}(b'_1) + \ell_{k_2}(b_2) \leq d$ and memory capacity permits, a single node can handle the computation of both S_1 and S_2 , and we allocate these two sessions to the same node. While the above discussion considers merging two sessions, the underlying principle generalizes to nodes containing multiple sessions.

Note that this algorithm does not assume the linear relationship between execution latency and batch size mentioned in Equation 1. The algorithm only assumes that the latency per input, $\ell(b)/b$, is non-decreasing with batch size b .

Finally, we extend the algorithm to be incremental across epochs, thus minimizing the movement of models across nodes. If the overall workload decreases, the scheduler attempts to move sessions from the least utilized backends to other backends. If a backend no longer executes any session, the scheduler releases the backend. If workload increases such that a backend becomes overloaded, we evict the cheapest sessions on this backend until it is no longer overloaded and perform bin packing again to relocate these evicted sessions.

6.2 Scheduling complex queries

We now present the query analysis algorithm that operates on dataflow representations of application queries in order to

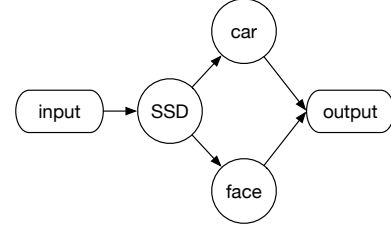


Figure 8: Dataflow graph of traffic analysis application.

determine the latency SLO splits for the constituent models. The output of this analysis is given as input to the scheduling algorithm of Section 6.1 that works with individual models.

Query analysis extracts the dataflow dependency graph between model invocations in application code. For example, Figure 8 depicts a traffic analysis application that first uses the SSD model to detect objects and recognizes cars and faces correspondingly. We formulate the scheduling of queries as the following optimization problem. Suppose the query involves a set of models M_i with request rate R_i , and the end-to-end latency SLO is L . The objective is to find the best latency SLO split L_i for each model M_i to minimize the total number of GPUs that are required for the query. Because latency L_i is determined by batch size b_i , the optimization problem is equivalent to finding the best batch sizes that minimizes GPU count, while meeting the latency SLO along every path from the root model (M_{root}) to the leaf models.

$$\begin{aligned} & \text{minimize}_{\{b_v\}} \sum_v R_v l_v(b_v)/b_v \\ & \text{subject to } \sum_{u: M_{\text{root}} \leadsto M_v} l_u(b_u) \leq L \quad \forall v \in \text{leaf} \end{aligned}$$

We use dynamic programming to solve this optimization problem for the case of fork-join dependency graphs, but limit our exposition to the simpler case of tree-like dependency graphs. For example, Figure 8 can be treated as a tree-structured dependency graph models (we can as the output does not invoke additional DNN models. Denote $f(u, t)$ as the minimum number of GPUs required to run models represented by u and the subtree at u within the time budget t . For a non-leaf node u , the algorithm allocates a time budget k for node u and at most $t - k$ for the rest of the subtree, and it then enumerates all $k \leq t$ to find the optimal split. More formally,

$$f(u, t) = \min_{k: k \leq t} \left\{ \min_{b: l_u(b) \leq k} R_u \frac{l_u(b)}{b} + \min_{t': t' \leq t - k} \sum_{v: M_u \leadsto M_v} f(v, t') \right\}$$

Since the dynamic programming cannot handle continuous state space, we approximate the state space of time budget with L/ϵ pieces of segments, where ϵ is the length of a segment. The time complexity is quadratic in L/ϵ .

6.3 Batch-aware dispatch

We now briefly describe the runtime mechanisms that control the execution of DNN tasks on backend nodes.

Overlapping CPU and GPU computation: DNN tasks can be decomposed into three stages: pre-processing (including decoding and batching images), forwarding, and post-processing. Pre- and post-processing are usually done on the CPU since they are not compute-intensive, whereas forwarding of a neural network model runs on the GPU. In order to achieve maximum GPU utilization, it is necessary to overlap the CPU computation and GPU computation. Therefore, the Nexus backend uses a thread pool of workers that pre-process the requests for the next batch and post-process the outputs of the previous batch on CPU, while another thread is dedicated to launching batched executions on the GPU. We observe experimentally that it usually takes 4 to 5 CPU cores to saturate GPU throughput, depending on the amount of computation in the models. Nexus uses an event-driven approach [28] to handling I/O, pre- and post-processing.

GPU Multiplexing: DNN frameworks provide no specific support for the concurrent execution of multiple models. For example, if two models that share a GPU execute in two processes or containers, they will independently issue requests to execute layers/kernels to the underlying GPU. The GPU runtime will typically serve these requests in first-come-first-served fashion, resulting in an arbitrary interleaving of the operations for the two models. The interleaving increases the execution latency of both models and makes it hard to predict the latency. Instead, the Nexus node runtime manages the execution of all models on a GPU, so it is able to pick batch sizes and execution schedules for all models in a round-robin fashion to make sure models abide by their latency SLOs. In addition, Nexus overlaps the pre- and post-processing in CPU with the GPU execution to increase GPU utilization.

Prefix Batching: Another important observation is that transfer learning [8, 26, 34, 36, 40] adapts a model from one dataset to another or from one task to another by re-training only the last few layers. DNN frameworks assume that if models differ in any layer, they cannot be executed in a batched fashion at all. However, in the common setting of model specialization, several models may differ only by their output layer. Batching the execution of all but the output layer can yield substantial batching gains. Nexus computes the hash of every sub-tree of the model schema and compares it with the existing models in the database to identify common sub-trees when a model is uploaded. At runtime, models with known common sub-trees are loaded partially in the backend and batched at the sub-tree (or prefix) granularity. The different suffix parts are then executed sequentially.

Adaptive Batching: As discussed in Section 4.3, lazy dropping during dispatch could lead to small batch sizes and low efficiency. In Nexus, we use an early drop policy that skips over requests that would cause sub-optimal batching. Specifically, the dispatcher scans through the queue using a sliding window whose length is the batch size determined by the

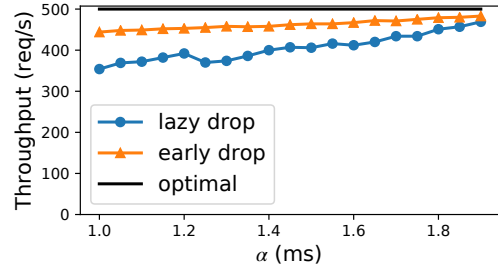


Figure 9: Maximal throughput achieved by lazy drop and early drop policy under various α .

global scheduler for a given session. It stops at the first request that has enough budget for batched execution latency of the entire window and drops all earlier requests. We use simulation to compare the lazy drop and early drop policy. Similar to Figure 5, we fix latency SLO to 100ms and optimal throughput to 500 req/s. Figure 9 depicts the throughput achieved by lazy drop and early drop policy under different α when 99% of requests are served within latency SLO. The results show that early drop can achieve up to 25% higher throughput than lazy drop.

7 Evaluation

We implemented Nexus in roughly 10k lines of C++ code. Nexus supports the execution of models trained by various frameworks including Caffe [18], Caffe2 [9], Tensorflow (TF) [1], and Darknet [32]. Nexus can be deployed in a cluster using Docker Swarm [7] (used below) or Kubernetes [11]. In our evaluation, we use this implementation to answer the following questions. (1) Does using Nexus result in better cluster utilization while meeting SLOs with respect to existing systems? (2) Does high performance persist when Nexus is used at a large scale? (3) How do the new techniques in Nexus contribute to its performance? (4) What determines how well each of these techniques work?

For a given workload and cluster, we refer to the maximum rate of queries that Nexus can process such that 99% of them are served within their latency SLOs as its *throughput*. We use throughput as the primary measure of cluster utilization.

7.1 Workload

Our basic approach is to run Nexus (and its various configurations and competitors) on either a small (16-GPU) or a large (100-GPU) cluster on various mixes of the applications and input videos specified in Table 4. These applications are modeled closely on widely-known video analysis scenarios, but we implemented each of them since we are unaware of freely available, widely used versions. They encompass a wide variety of characteristics. Some (e.g., game and traffic, which implements Figure 8) are based on 24/7 live video streams, whereas others (e.g., dance and logo) apply to footage of individual performances. Some require simple queries (e.g.,

name	brief description	models used	video input	Nexus features used
game	analyze streamed video games	text, object rec.	Twitch [37] streams, 1 week, 50 streamers	SS, ED, QA-1, PB
traffic	surveil traffic on streets	object det., face rec.	traffic cameras, 1 week, 20 cameras	SS, ED, QA-2
dance	rate dance performances	person det., pose rec.	dance videos from YouTube, 2 hrs	SS, ED, QA-2
bb	gauge response to public billboard	person, face det., gaze, age, sex rec.	game show audience videos, 12 hours	SS, ED, QA-3, PB
bike	find bike-rack occupancy on buses	object, text det./rec./trk.	traffic cameras, 1 week, 10 cameras	SS, ED, QA-4, PB
amber	match vehicle to "Amber Alert" description	object det., car make+model rec., text det./rec.	dashcam videos from YouTube, 12 hours	SS, ED, QA-4, PB
logo	audit corporate logo placement	person icon, pose, text, person det./rec.	NFL, NBA game videos, 24 hours	SS, ED, QA-5, PB

Table 4: Evaluated application and input data. Squishy scheduling, early drop, complex query analysis and prefix-batching are abbreviated as SS, ED, QA and PB. QA- k indicates that the related complex query has k stages. Models for detection, recognition and tracking are abbreviated 'det.', 'rec.' and 'trk.'

game, designated "QA-1" has 1 stage), and others more complex ones (e.g., the 5-stage logo, designated "QA-5", seeks to detect people, find their torsos, look for logos, and if found, detect and recognize the player's number). Most use multiple specialized versions of models and are therefore amenable to prefix batching, designated "PA". For each workload, we have collected several hours and many streams (for live streams) or files (for episodes) of video, which sample and play in a loop to preserve temporal characteristics while allowing arbitrarily long simulations. Unless otherwise mentioned, we sample inter-arrival time between frames uniformly.

7.2 Using Clipper and Tensorflow as baselines

Clipper and TF Serving assume cluster scheduling and latency SLOs for DNN invocations are handled externally. Careful scheduling and latency allocation are two of Nexus's core contributions. To provide a basis for comparison, we furnish simple default versions of each. A *batch-oblivious* scheduler² greedily allocates to each model/SLO a share of the cluster proportional to its request rate and inversely proportional to its maximum single-node throughput. Further, we split the latency for a query evenly across its stages. The oblivious scheduler may map multiple models onto a Clipper GPU, in which case we launch one container per model on the GPU. We rely on Clipper's load balancer to manage

²Note that we retain Clipper's adaptive *batching*, which is orthogonal to the *scheduling* scheme used, in all experiments. Adaptive batching groups requests into batches on a single backend node, adapting the batch size dynamically to get higher throughput. Scheduling, on the other hand, works at cluster-scale over a coarse epoch granularity and performs resource allocation. In particular, it determines how many replicas to use for each model, and which GPUs to place them on. Clipper assumes an external scheduler, so we had to provide a batch-oblivious scheduler as a reasonable baseline.

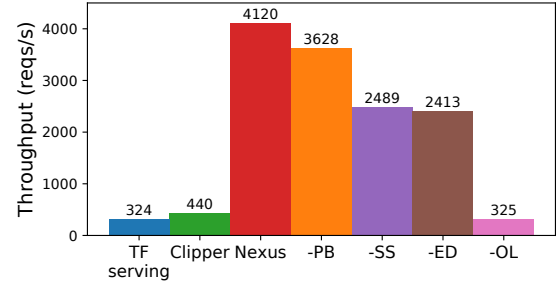


Figure 10: Game analysis ablation study.

model replicas. In contrast, TF does not provide a frontend load balancer, nor does it allow the specification of latency SLOs per request. We, therefore, provide a dispatcher and pick the maximum batch size for each model, so its SLO is not violated.

7.3 Single-application case studies

To compare our performance with those of Clipper and TF Serving, we ran the game and traffic applications separately on a 16-GPU cluster. In each case, we ran an ablation study on Nexus features to gauge their impact.

7.3.1 Game analysis

When analyzing game streams, we seek to recognize six numbers (e.g., number of kills, number of players alive) and one object icon on each frame. We use versions of LeNet [22] specialized to the game's font and the number of digits to recognize numbers, and ResNet-50 [15] with its last layer specialized to recognize the icon. We include 20 games in the case study, and consider a latency SLO of 50ms (sensitivity to SLOs is analyzed in Section 7.5). The request rates of frames from the 20 games follow the Zipf-0.9 distribution. We noticed that both Clipper and TF show extremely poor throughput on the tiny LeNet model. We conjecture, but could not confirm, that this is because of inadequate parallelism between CPU and GPU processing. To be maximally fair to them, we allow the two baselines to invoke just the ResNet model. Their resulting throughput, which we report, is better by over 4× than including LeNet. Finally, we additionally turn off prefix batching (PB), squishy scheduling (SS), early drop (ED), and overlapped processing in the CPU and GPU (OL, see Section 6.3). The game query has only 1 stage and, therefore, does not exercise query analysis (QA).

Figure 10 shows the results. **Nexus increases throughput significantly**, by 9.4 and 12.7× relative to Clipper and TF Serving on this application. **Several of Nexus's techniques contribute**, with OL the most (incrementally disabling OL results in an additional 7.4× throughput reduction), and ED the least (disabling it results in a 3% reduction). Note that even though OL is the dominant technique for this application, the other Nexus techniques together result in a 48% fall in throughput (from 4120 to 2143 req/s) when disabled. ED is designed to address variability in requests. When we

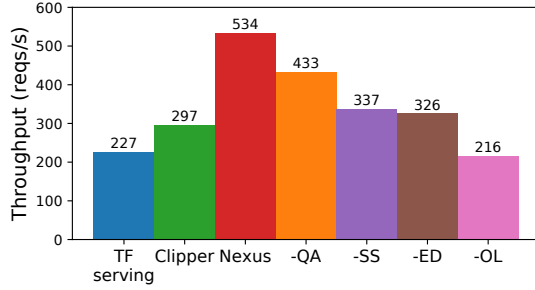


Figure 11: Traffic analysis ablation study.

re-ran the experiments with frame inter-arrival rates sampled from a Poisson distribution as opposed to uniform, the significance of ED increases, and throughput drops by a more significant 8.5%.

Disabling OL causes a dramatic reduction in throughput for this application because of a complex interaction between its tight (50ms) SLO, relatively high preprocessing times (roughly 10ms), and the low forwarding times of the models (roughly 6ms for ResNet-50 and under 0.1ms for LeNet). Serializing preprocessing with GPU execution for each batch results in roughly half the cycles of the GPU remaining idle. Further, given the tight SLO and the latency of preprocessing, batch sizes need to remain close to 1 to guarantee timely execution. These small batch sizes have many times lower throughput than the optimal large batches. **Taken together, these factors result in a large throughput loss, making OL critical in the tight-SLO/small model regime.** As the analysis for the traffic monitoring application (Figure 11) below shows, with more relaxed SLOs and larger models, the importance of overlapped preprocessing (OL) is diminished.

7.3.2 Traffic monitoring

traffic uses SSD [4], VGG-Face [29] and GoogleNet-car [39] for object detection, face recognition and car make/model analysis on 20 long-running traffic streams with a latency SLO of 400ms. These models are larger than those used in the game analysis case: the largest (SSD), which is invoked on every frame, runs for 47ms at batchsize 1 and the smallest (Googlenet) runs for 4.2ms, compared to 6.2ms/0.1ms for gaming.

Our first experiment replicates the cumulative ablation test that we performed on gaming analysis in the previous section. Figure 11 shows the results for analyzing non-rush-hour traffic data.³ **Once again, maximum throughput comes from many Nexus techniques working together.** All techniques other than OL, when disabled together, result in a significant 39% performance drop relative to full Nexus (throughput falls from 534 to 326 req/s). Further, the contribution of ED is small.

On the other hand, **the relative contribution of OL is**

³See the analysis below comparing rush-hour with non-rush-hour results.

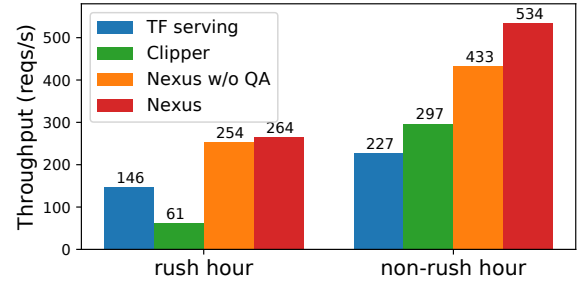


Figure 12: Diurnal throughput variation for traffic analysis.

much smaller in this application: disabling OL incrementally results in throughput falling only by a further 34% (from 326 to 216 req/s), as opposed to the previous 7.4× drop. Through more detailed analysis, we confirm that the more relaxed SLO enables large-batch execution, which together with the large model forwarding time renders the preprocessing time relatively small (so less of the GPU stays idle) and allows high throughput due to batch execution.

Unlike game, traffic is a two-stage application: the first stage detects vehicles or people, and the second stage recognizes their make/model or identity. Query Analysis (QA) is thus applicable. Instead of splitting latency evenly (which is the baseline, represented by -QA the figure), QA allocates 345ms of the 400ms latency to object detection via SSD. **Disabling QA results in a significant throughput loss**, with throughput falling by 19% (from 534 to 433 req/s).

An interesting aspect of the traffic application is that the nature of data analysis depends strongly on whether the traffic being analyzed is rush-hour. Figure 12 summarizes throughput achieved on non-rush-hour vs rush-hour traffic. Three points are worth noting. First, throughput achieved during rush hour is significantly less than that during non-rush hour. This is because rush-hour traffic is more complex: more vehicles are detected, and require follow-on analysis, on every frame. Second, although during rush hour, the benefit of Nexus relative to the TF Serving baseline falls (from $534/227 = 2.4\times$ during non-rush hour to $264/146 = 1.8\times$ during rush hour)⁴, **the benefit provided by Nexus is still significant.** In particular, the relative benefit of the QA technique also falls. This seems to be because various subsystems are over-subscribed during rush hour.

7.4 Long-running multi-application deployment

To check whether Nexus maintains its gains when run at large scale, especially in the face of significant workload variation across multiple applications, we deployed Nexus on a cluster ranging from 16 to 100 GPUs on a commercial cloud, running all applications from Table 4 simultaneously for a period of several hours. We focused on two metrics. First, how close to optimal is GPU utilization during this

⁴We were unable to determine why the relative performance of Clipper fell so sharply on rush-hour data.

period? Second, how well does Nexus react to workload changes?

We first study the optimality of Nexus by using the uniform distribution to generate highly controlled workloads for the applications and perform the evaluation on a cluster of 16 GTX 1080Ti GPUs. To bound the optimal (smallest) number of GPUs needed for a session, we assumed that its model is fully (not just prefix) batchable, that its SLO allows it to run at the optimal batch size, and that it has enough requests coming in to be scheduled back-to-back on GPUs. Of course, real sessions often violate one or more of these assumptions and will have lower throughput. Nexus achieved a bad rate of less than 1% consistently and used 11.7 GPUs on average. We then computed the theoretical lower bound of the number of GPUs required based on the maximal throughput that a model can achieve on a single GPU. The lower bound for this workload is 9.8 GPUs on average. It indicates that Nexus scheduler can achieve 84% of GPU efficiency compared to the theoretical lower bound.

Next, we deploy Nexus on 100 K80s and evaluate workloads with varying Poisson arrival rates. In particular, we fixed the number of model sessions, designated by a given model and its latency SLO, per application (e.g., game had 50 model sessions, traffic had 20), but varied the request rate per session by varying the rate at which each submitted frames.

Figure 13 shows Nexus adapting to a change in workload during a 1000-sec window of the deployment. The top panel shows a stacked graph of requests over time, the middle one the number of GPUs allocated and the bottom one the bad rate, with instantaneous bad rates above 1% marked in red. Around 326s into the window, the number of requests increases and starts varying significantly. Nexus, which is running with 30s epochs, starts dropping requests, detects the change within 12s (this could have been as long as 30s) and allocates more GPUs. It deallocates GPUs (this time with a roughly 10s lag) at the 644s mark when demand subsides. Nexus violates the latency SLOs on 0.27% of requests on average. The sporadic high bad rate (>1%) is mainly due to scheduling reconfiguration triggered by workload changes.

These results illustrate that **Nexus responds well to variable workloads at large scale and is able to allocate close to the aggressive theoretical lower bound.**

7.5 Sensitivity analysis of Nexus features

We now present micro-benchmarks to analyze the main components of Nexus. Overall, we find that Nexus’s core techniques are quite robust to variations in key design parameters.

GPU Multiplexing. The Nexus runtime (Section 6.3) focuses on minimizing interference on GPU between executing models (by avoiding interleaving during their execution), and idling while switching between models (by overlapping pre/post-processing on CPU with model execution on the

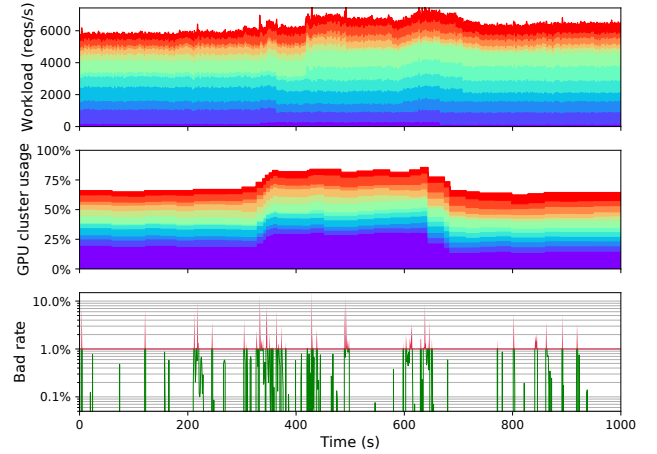


Figure 13: A 1000 sec window from our large-scale deployment.

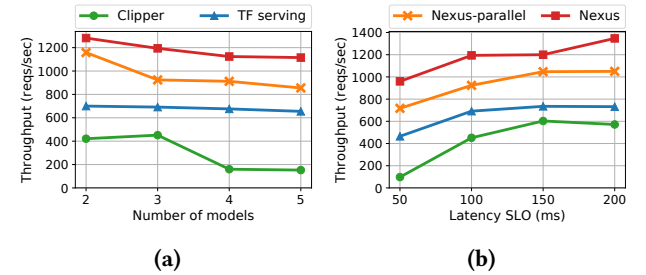


Figure 14: Impact on throughput of varying numbers of models served (a) and latency SLOs (b) under GPU multiplexing.

GPU, and not waiting for fixed target batch sizes to fill up before dispatch to the GPU).

Figure 14 analyzes the importance of these features by comparing the throughput of Nexus with those of Clipper, TF Serving, and a version of Nexus (“Nexus-parallel”) that issues models in parallel and does not control interference. This experiment runs increasing numbers of copies of the Inception model with a latency SLO of 100ms. Throughputs of all four models suffer, TF Serving less than Clipper because it runs models in a round-robin fashion whereas Clipper deploys them in independent containers that interfere. Nexus achieves 1.4–2.1× throughput compared to TF serving, and 1.9–9.8× throughput compared to Clipper on a single GPU. Nexus-parallel fares better because it avoids idling (but still suffers from interference), and Nexus fares the best. We see similar trends across other models. Figure 14(b) compares the throughput while varying the latency SLO from 50ms to 200ms, with the number of models fixed at 3. When latency SLO becomes higher, the greater scheduling slack gives Nexus-parallel higher throughput.

Prefix Batching. Figure 15 examines how the throughput and memory benefits of prefix batching scale as the number of variants of Resnet50 that differ only in the last layer increases, on a single GPU. Figure 15(a), compares prefix

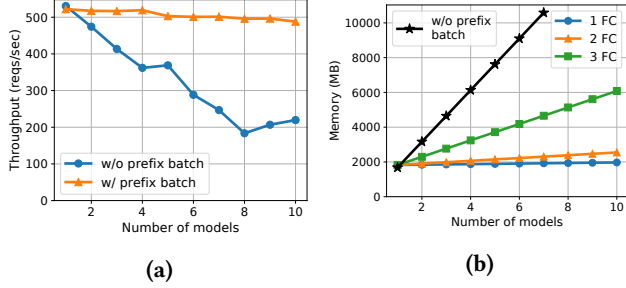


Figure 15: Impact on throughput (a) and memory use (b) of varying numbers of batched models under prefix batching.

batching to unbatched execution of the variants. Without prefix batching, the variants have to execute on smaller "sub-batches" to satisfy their SLOs, yielding worse aggregate throughput. With prefix batching, since many models can execute in one batch, the sub-batches can be aggregated into large batches that maintain up to 110% higher throughput.

Similarly, when the (unshared) model suffixes are small ("1 FC", indicating one "fully connected" unshared layer, in Figure 15(b)), additional model variants use negligible extra GPU memory. As the number of unshared layers increase ("2 FC" and "3 FC" add two and three fully connected layers to the shared prefix), the memory benefits fall. Without prefix batching (black line), however, we quickly run out of GPU memory even if a model has only one unshared layer.

Squishy Scheduling. We now examine the sensitivity of squishy scheduling to model types, request rates, and SLOs. We compare the throughput of Nexus with squishy scheduling to a baseline version of Nexus that uses batch-oblivious scheduling instead. Both need to allocate 16 sessions on 8 GPUs under 5 scenarios: (a) Inception or (b) ResNet models with mixed SLOs ranging from 50ms to 200ms, (c) Inception or (d) ResNet models with mixed request rates following Zipf-0.9 distribution, (e) 8 different model architectures, each associated with two SLOs, 50ms and 100ms. Figure 16 depicts the relative throughput of standard Nexus with regard to baseline. Nexus outperforms baseline across all mixes, with the highest gains (up to 64%) coming from handling varying request rates, and the lowest (11%) coming from handling varying request mixes.

Complex Query Analysis. To evaluate the performance gain of the query analyzer, we compare the throughput of Nexus with and without the query analyzer. The baseline simply splits the latency SLO evenly across the various stages in the query. The query includes two stages: (a) the first stage executes SSD, and then (b) invokes Inception model for γ times. The experiment is performed on 8 GPUs. We vary the latency SLO from 300ms to 500ms and choose γ to be 0.1, 1, and 10. Figure 17 shows that Nexus with the query analyzer achieves 13–55% higher throughput than the baseline.

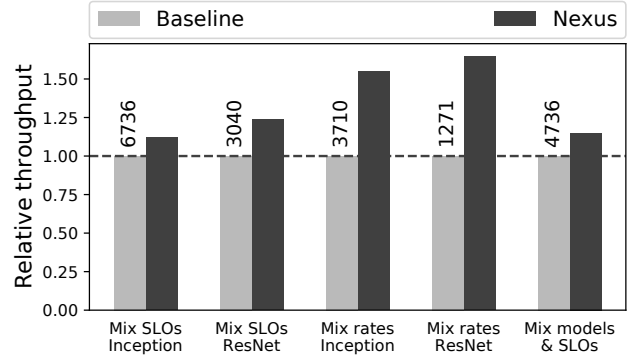


Figure 16: Impact on throughput of varying model- and SLO-mixes under squishy scheduling.

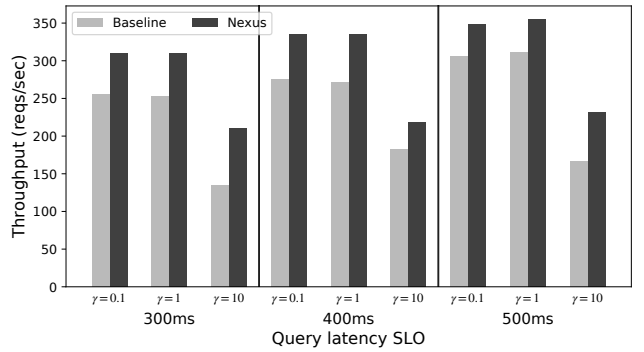


Figure 17: Impact on throughput of varying query latency SLO and γ (see Section 4.2) under complex query analysis.

8 Conclusion

We present a scalable and efficient system design for serving Deep Neural Network (DNN) applications. Instead of serving the entire application in an opaque CPU-based container with models embedded in it, which leads to sub-optimal GPU utilization, our system operates directly on models and GPUs. This design enables several optimizations in batching and allows more efficient resource allocation. Our system is fully implemented, in C++, and evaluation shows that Nexus can achieve 1.8-12.7 $\times$ more throughput relative to state-of-the-art baselines while staying within latency constraints (achieving a "good rate") over 99% of the time.

Acknowledgements

We thank Lenin Sivalingam, Peter Bodik, anonymous reviewers, and our shepherd, Ion Stoica, for their helpful feedback. We also thank the Watch For team at Microsoft Research for insights and suggestions on game stream analysis and large-scale video processing. This work was supported by NSF award CNS-1614717.

References

- [1] Martin Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek Gordon Murray, Benoit Steiner, Paul A. Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: A System for Large-Scale Machine Learning. In *12th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2016, Savannah, GA, USA, November 2-4, 2016*. USENIX Association, 265–283.
- [2] Ahmad Abdelfattah, Azzam Haidar, Stanimire Tomov, and Jack Dongarra. 2017. Fast Cholesky factorization on GPUs for batch and native modes in MAGMA. *Journal of Computational Science* 20 (2017), 85–93.
- [3] Atul Adya, Daniel Myers, Jon Howell, Jeremy Elson, Colin Meek, Vishesh Khemani, Stefan Fulger, Pan Gu, Lakshminath Bhuvanagiri, Jason Hunter, Roberto Peon, Larry Kai, Alexander Shraer, Arif Merchant, and Kfir Lev-Ari. 2016. Slicer: Auto-Sharding for Datacenter Applications. In *12th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2016, Savannah, GA, USA, November 2-4, 2016*. USENIX Association, 739–753.
- [4] Liang-Chieh Chen, George Papandreou, Iasonas Kokkinos, Kevin Murphy, and Alan L Yuille. 2018. Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs. *IEEE transactions on pattern analysis and machine intelligence* 40, 4 (2018), 834–848.
- [5] Clipper contributors. 2019. Clipper v0.4. <https://github.com/ucbrise/clipper/tree/release-0.4>.
- [6] Daniel Crankshaw, Xin Wang, Giulio Zhou, Michael J. Franklin, Joseph E. Gonzalez, and Ion Stoica. 2017. Clipper: A Low-Latency Online Prediction Serving System. In *14th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2017, Boston, MA, USA, March 27-29, 2017*. USENIX Association, 613–627.
- [7] Docker. 2014. Docker Swarm. <https://github.com/docker/swarm>.
- [8] Jeff Donahue, Yangqing Jia, Oriol Vinyals, Judy Hoffman, Ning Zhang, Eric Tzeng, and Trevor Darrell. 2014. DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition. In *Proceedings of the 31th International Conference on Machine Learning, ICML 2014, Beijing, China, 21-26 June 2014 (JMLR Workshop and Conference Proceedings)*, Vol. 32. JMLR.org, 647–655.
- [9] Facebook. 2018. Caffe2: A New Lightweight, Modular, and Scalable Deep Learning Framework. <https://caffe2.ai/>.
- [10] Michael R. Garey and David S. Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman & Co., New York, NY, USA.
- [11] Google. 2014. Kubernetes: Production-Grade Container Orchestration. <https://kubernetes.io/>
- [12] Google. 2019. Cloud AutoML Vision. <https://cloud.google.com/vision/automl/docs/>.
- [13] Juncheng Gu, Mosharaf Chowdhury, Kang G Shin, Yibo Zhu, Myeongjae Jeon, Junjie Qian, Hongqiang Liu, and Chuanxiong Guo. 2019. Tiresias: A GPU Cluster Manager for Distributed Deep Learning. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. 485–500.
- [14] Seungyeop Han, Haichen Shen, Matthai Philipose, Sharad Agarwal, Alec Wolman, and Arvind Krishnamurthy. 2016. MCDNN: An approximation-based execution framework for deep stream processing under resource constraints. In *Proceedings of the 14th Annual International Conference on Mobile Systems, Applications, and Services*. ACM, 123–136.
- [15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.
- [16] Benjamin Hindman, Andy Konwinski, Matei Zaharia, Ali Ghodsi, Anthony D. Joseph, Randy H. Katz, Scott Shenker, and Ion Stoica. 2011. Mesos: A Platform for Fine-Grained Resource Sharing in the Data Center. In *Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2011, Boston, MA, USA, March 30 - April 1, 2011*. USENIX Association.
- [17] Kevin Hsieh, Ganesh Ananthanarayanan, Peter Bodik, Shivaram Venkataraman, Paramvir Bahl, Matthai Philipose, Phillip B. Gibbons, and Onur Mutlu. 2018. Focus: Querying Large Video Datasets with Low Latency and Low Cost. In *13th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2018, Carlsbad, CA, USA, October 8-10, 2018*. USENIX Association, 269–286.
- [18] Yangqing Jia, Evan Shelhamer, Jeff Donahue, Sergey Karayev, Jonathan Long, Ross Girshick, Sergio Guadarrama, and Trevor Darrell. 2014. Caffe: Convolutional architecture for fast feature embedding. In *Proceedings of the 22nd ACM international conference on Multimedia*. ACM, 675–678.
- [19] Angela H Jiang, Daniel L-K Wong, Christopher Canel, Lilia Tang, Is-han Misra, Michael Kaminsky, Michael A Kozuch, Padmanabhan Pillai, David G Andersen, and Gregory R Ganger. 2018. Mainstream: Dynamic stem-sharing for multi-tenant video processing. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. 29–42.
- [20] Daniel Kang, John Emmons, Firas Abuzaid, Peter Bailis, and Matei Zaharia. 2017. NoScope: optimizing neural network queries over video at scale. *Proceedings of the VLDB Endowment* 10, 11 (2017), 1586–1597.
- [21] Bernhard Korte and Jens Vygen. 2008. *Bin-Packing. In Combinatorial Optimization: Theory and Algorithms*. Springer Berlin Heidelberg, Berlin, Heidelberg, 449–465. https://doi.org/10.1007/978-3-540-71844-4_18
- [22] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324.
- [23] Microsoft. 2018. Virtual Machine Scale Sets. <https://docs.microsoft.com/en-us/azure/virtual-machine-scale-sets/virtual-machine-scale-sets-overview>.
- [24] Microsoft. 2019. Custom Vision. <https://azure.microsoft.com/en-us/services/cognitive-services/custom-vision-service/>.
- [25] Christopher Olston, Noah Fiedel, Kiril Gorovoy, Jeremiah Harmsen, Li Lao, Fangwei Li, Vinu Rajashekhar, Sukriti Ramesh, and Jordan Soyke. 2017. Tensorflow-serving: Flexible, high-performance ml serving. *arXiv preprint arXiv:1712.06139* (2017).
- [26] Maxime Oquab, Leon Bottou, Ivan Laptev, and Josef Sivic. 2014. Learning and transferring mid-level image representations using convolutional neural networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 1717–1724.
- [27] Kay Ousterhout, Patrick Wendell, Matei Zaharia, and Ion Stoica. 2013. Sparrow: distributed, low latency scheduling. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*. ACM, 69–84.
- [28] Vivek S Pai, Peter Druschel, and Willy Zwaenepoel. 1999. Flash: An efficient and portable Web server. In *USENIX Annual Technical Conference, General Track*. 199–212.
- [29] Omkar M. Parkhi, Andrea Vedaldi, and Andrew Zisserman. 2015. Deep Face Recognition. In *Proceedings of the British Machine Vision Conference 2015, BMVC 2015, Swansea, UK, September 7-10, 2015*. BMVA Press, 41.1–41.12. <https://doi.org/10.5244/C.29.41>
- [30] Yanghua Peng, Yixin Bao, Yangru Chen, Chuan Wu, and Chuanxiong Guo. 2018. Optimus: an efficient dynamic resource scheduler for deep learning clusters. In *Proceedings of the Thirteenth EuroSys Conference*. ACM, 3.
- [31] Mohammad Rastegari, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. 2016. Xnor-net: Imagenet classification using binary convolutional neural networks. In *European Conference on Computer Vision*. Springer, 525–542.
- [32] Joseph Redmon. 2013–2016. Darknet: Open Source Neural Networks in C. <http://pjreddie.com/darknet/>.

- [33] Malte Schwarzkopf, Andy Konwinski, Michael Abd-El-Malek, and John Wilkes. 2013. Omega: flexible, scalable schedulers for large compute clusters. In *Proceedings of the 8th ACM European Conference on Computer Systems*. ACM, 351–364.
- [34] Ali Sharif Razavian, Hossein Azizpour, Josephine Sullivan, and Stefan Carlsson. 2014. CNN features off-the-shelf: an astounding baseline for recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*. 806–813.
- [35] Haichen Shen, Seungyeop Han, Matthai Philipose, and Arvind Krishnamurthy. 2017. Fast video classification via adaptive cascading of deep models. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 3646–3654.
- [36] Karen Simonyan and Andrew Zisserman. 2014. Two-stream convolutional networks for action recognition in videos. In *Advances in neural information processing systems*. 568–576.
- [37] Twitch. 2011. Twitch. <https://www.twitch.tv/>.
- [38] Wencong Xiao, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng, Hanyu Zhao, Quanlu Zhang, Fan Yang, and Lidong Zhou. 2018. Gandiva: Introspective Cluster Scheduling for Deep Learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, 595–610.
- [39] Linjie Yang, Ping Luo, Chen Change Loy, and Xiaoou Tang. 2015. A large-scale car dataset for fine-grained categorization and verification. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 3973–3981.
- [40] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. 2014. How transferable are features in deep neural networks?. In *Advances in neural information processing systems*. 3320–3328.
- [41] Dong Yu, Frank Seide, Gang Li, and Li Deng. 2012. Exploiting Sparseness In Deep Neural Networks For Large Vocabulary Speech Recognition. In *Proceedings of IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*.
- [42] Haoyu Zhang, Ganesh Ananthanarayanan, Peter Bodik, Matthai Philipose, Paramvir Bahl, and Michael J Freedman. 2017. Live video analytics at scale with approximation and delay-tolerance. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. 377–392.

A Hardness of Fixed-rate GPU Scheduling Problem (FGSP)

We now justify the use of an approximate algorithm for GPU cluster scheduling. We define the *Fixed-rate GPU Scheduling Problem* (FGSP), which is a highly restricted version of the general problem, and we show that even the restricted version is intractable.

FGSP:

Input - models M_i , $1 \leq i \leq n$ with corresponding latencies L_i , latency bounds B_i and GPU count C . (The latencies correspond to the fixed rates.)

Output - Partition of the models into C sets so that in each set S we have $D + L_i \leq B_i, \forall i \in S$ where $D = \sum_{i \in S} L_i$ is the duty cycle for the set.

We show that FGSP is strongly NP-hard by reduction from 3-PARTITION [10].

Theorem 1. *FGSP is strongly NP-complete.*

Proof. We start with a given instance of 3-PARTITION which consists of a bound B and $3n$ $\frac{B}{4} \leq a_1, a_2, \dots, a_{3n} \leq \frac{B}{2}$; the goal of 3-PARTITION is to partition the a_i s into triples such that the sum of each triple is B . Observe that wlog we may assume that $\sum_{1 \leq i \leq 3n} a_i = nB$.

From the given instance of 3-PARTITION we create an instance of FGSP by setting $L_i = 2B + a_i, B_i = 9B + a_i, \forall 1 \leq i \leq 3n, C = n$.

It is clear that if there exists a solution to the 3-PARTITION instance then the same partition into n triples yields a partition of the FGSP instance into $C = n$ sets so that $D + L_i \leq 9B + a_i$ since $D = 7B$ and $L_i = 2B + a_i$. In the other direction suppose there exists a solution to FGSP. Observe that in any solution to FGSP every set can have at most 3 models because otherwise the duty cycle D would exceed $8B$ and then the constraint $D + L_i \leq B_i$ would be violated for any i in the set, since $D + L_i > 10B$ but $B_i < 10B$. Since there are a total of $3n$ models and $C = n$ sets every set must have exactly 3 models, i.e. every set must be a triple. Since $D + L_i \leq B_i$ for any i in the set, we have that $D + 2B + a_i \leq 9B + a_i$ or $D \leq 7B$. But this implies that in every triple the sum of the L_i s is at most $7B$ or the sum of the corresponding a_i s is at most B . But since the sum of all the n triples is nB and each triple is at most B it must be that the sum of each triple is exactly B . This means that the partition of models of the FGSP instance into sets is also a solution for the partition of the corresponding a_i into triples in 3-PARTITION. $\square$